

Set Abacus

1. The cache table will update after every memory access request.

a.

Index	Valid bit	Tag
000	1	100
001	0	
010	1	111 111
011	1	001
100	1	100
101	1	000
110	1	010 111
111	0	

100100	miss
111010	miss (as valid bit is zero)
010110	hit
100100	hit
111110	miss

b. hit ratio = $\frac{25}{60} = 0.4$

miss ratio = $\frac{35}{60} = 0.6$

c. To balance speed, cost, and capacity—small, fast, expensive memories (like cache) are placed close to the CPU, while larger, slower, cheaper memories (like main memory and storage) are placed farther away, improving overall system performance.

Disadvantage of adding more layers:

It increases design complexity and can add extra access latency and management overhead (e.g., cache coherence and control logic).

2. $2^{12} (12 + 1 + 8) = 4452$ bits
3. Data is written to both cache and main memory on every write.

Problem:

High memory write traffic → slower performance.

Solution:

Use a write buffer to temporarily hold writes and reduce CPU stalls.

Set Byte

1. The cache table will update after every memory access request.

a.

Index	Valid bit	Tag
000	1	100
001	0	
010	1	111
011	1	001
100	1	100
101	1	000
110	1	010 , 010 , 111
111	0	

100100	miss
111010	hit
010110	miss (as valid bit 0)
100100	hit
111110	miss

b. hit ratio = $\frac{25}{60} = 0.4$

miss ratio = $\frac{35}{60} = 0.6$

c. To balance speed, cost, and capacity—small, fast, expensive memories (like cache) are placed close to the CPU, while larger, slower, cheaper memories (like main memory and storage) are placed farther away, improving overall system performance.

Disadvantage of adding more layers:

It increases design complexity and can add extra access latency and management overhead (e.g., cache coherence and control logic).

2. $2^{12} (12 + 1 + 8) = 4452$ bits
3. Data is written only to cache; main memory is updated when the block is replaced.

Problem:

Main memory may have stale (inconsistent) data.

Solution:

Use a dirty (modified) bit to track changed blocks and update memory on eviction.